Deep Learning

Practical Exam Notes

Practical 1: ANN for Boston House Price Prediction
Practical 2: CNN for Fashion MNIST Classification
Practical 3: RNN for Google Stock Price Prediction

PRACTICAL 1: ANN for Boston House Price Prediction

🎯 Aim / Objective

To build an Artificial Neural Network (ANN) using Keras to predict the median house price (MEDV) of Boston suburbs based on features like crime rate, number of rooms, and pupil-teacher ratio.

📚 Theory

What is an Artificial Neural Network (ANN)?

An Artificial Neural Network (ANN) is a computational model inspired by biological neural networks in the human brain. It consists of layers of interconnected nodes (neurons) that process information. ANNs learn from data by adjusting connection weights through a process called backpropagation and gradient descent.

An ANN has three main types of layers:

- Input Layer: Receives raw input features (in this case, 3 housing features).
- Hidden Layers: Intermediate layers that learn complex representations of the data using activation functions.
- Output Layer: Produces the final prediction. For regression, it typically has 1 neuron with a linear activation.

Key Concepts Used in This Practical

- Regression Problem: We are predicting a continuous numerical value (house price in $1000s), not a category. This is called a regression task.
- ReLU Activation (Rectified Linear Unit): f(x) = max(0, x). Used in hidden layers to introduce non-linearity. It prevents the vanishing gradient problem and is computationally efficient.
- Linear Activation: Used in the output layer for regression. It allows the output to take any real number value.
- Adam Optimizer: Adaptive Moment Estimation. It combines the advantages of RMSProp and Momentum optimizers. It is fast, efficient, and requires minimal tuning.
- MSE Loss (Mean Squared Error): Loss = (1/n) * Σ(y_actual - y_pred)². Used for regression problems. Penalizes large errors more heavily.
- MAE (Mean Absolute Error): MAE = (1/n) * Σ|y_actual - y_pred|. A more interpretable metric — tells the average error in $1000s.
- StandardScaler: Normalizes features to have mean=0 and standard deviation=1. This helps the neural network train faster and converge better.

Boston Dataset Features

Feature	Description
LSTAT	% of lower-status population — negatively correlated with price
PTRATIO	Pupil-teacher ratio by town — more students per teacher = lower price
RM	Average number of rooms per dwelling — positively correlated with price
MEDV (Target)	Median value of owner-occupied homes in $1000s

🔢 Step-by-Step Code Explanation

Step 1: Import Libraries & Load Data

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv('Boston.csv')
df.drop(columns=['B'], inplace=True)   # Drop 'B' column (racial demographic)
df.isnull().sum()                       # Check for missing values
df.describe()                           # Statistical summary
df.corr()['MEDV'].sort_values()         # Correlation with target
```

We first load the dataset and explore it. The 'B' column is dropped as it represents a racial demographic factor. We then check for null values and look at the correlation of each feature with the target MEDV.

Step 2: Feature Selection & Train-Test Split

```
X = df.loc[:, ['LSTAT', 'PTRATIO', 'RM']]   # Top correlated features
Y = df.loc[:, 'MEDV']                         # Target variable

from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(
    X, Y, test_size=0.25, random_state=10
)
```

We select 3 most correlated features: LSTAT (negative correlation), PTRATIO (negative), and RM (positive). 75% data is used for training and 25% for testing.

Step 3: Feature Scaling

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(x_train)                    # Fit only on training data
x_train = scaler.transform(x_train)   # Transform training set
x_test  = scaler.transform(x_test)    # Transform test set (same scale)
```

StandardScaler is fitted ONLY on training data to avoid data leakage. Formula: z = (x - mean) / std. This standardizes each feature so the model trains efficiently.

Step 4: Build the ANN Model

```
from keras.models import Sequential
from keras.layers import Dense, Input

model = Sequential()
model.add(Input(shape=(3,)))              # 3 input features
model.add(Dense(128, activation='relu'))  # Hidden layer 1: 128 neurons
model.add(Dense(64, activation='relu'))   # Hidden layer 2: 64 neurons
model.add(Dense(1, activation='linear'))  # Output: 1 value (price)

model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.summary()
```

The model has 3 layers. The input shape is (3,) because we have 3 features. The two hidden layers use ReLU activation. The output layer has 1 neuron with linear activation since we are predicting a continuous value (house price).

Step 5: Train & Evaluate

```
model.fit(x_train, y_train, epochs=100, validation_split=0.05)

output = model.evaluate(x_test, y_test)
print(f'MSE: {output[0]}')
print(f'MAE: {output[1]}')

y_pred = model.predict(x_test)
print(*zip(y_pred, y_test))
```

📝 Example

Practical Example — Boston Housing

Suppose a suburb has: LSTAT=5 (5% low-status), PTRATIO=15 (15 pupils/teacher), RM=7 (7 rooms avg). After standardization and passing through the ANN, the model might predict MEDV ≈ $32,000. A suburb with LSTAT=30, PTRATIO=22, RM=4 might predict MEDV ≈ $10,000 — showing that more rooms, fewer low-status residents, and lower pupil-teacher ratio leads to higher house prices.

🗣️ Oral Questions & Answers — Practical 1

Q: What is a neural network?

A: A neural network is a machine learning model inspired by the human brain. It consists of layers of neurons (nodes) connected with weights. It learns by adjusting these weights using backpropagation to minimize error.

Q: What is the difference between regression and classification?

A: Regression predicts a continuous output (e.g., house price = $32,000). Classification predicts a discrete class (e.g., cat or dog). ANN-1 uses regression; ANN-2 uses classification.

Q: Why use StandardScaler before training?

A: Neural networks are sensitive to the scale of inputs. If one feature has values in 1000s and another in 0-1 range, the optimizer struggles. StandardScaler makes all features have mean=0 and std=1, helping the model train faster and converge properly.

Q: Why is ReLU used in hidden layers?

A: ReLU (f(x) = max(0,x)) introduces non-linearity, allowing the network to learn complex patterns. It also avoids the vanishing gradient problem that older activations like sigmoid had. It's computationally simple and fast.

Q: Why MSE as loss for regression?

A: Mean Squared Error penalizes large errors more than small ones due to squaring. It is differentiable (needed for gradient descent) and works well for continuous predictions. The optimizer minimizes this during training.

Q: What is train_test_split and why 75-25?

A: It divides the dataset into training (75%) and testing (25%) sets. The model learns from training data and is evaluated on test data (unseen during training) to check generalization. random_state=10 ensures reproducibility.

Q: What is the Adam optimizer?

A: Adam (Adaptive Moment Estimation) is an optimization algorithm that combines RMSProp and Momentum. It adapts the learning rate for each parameter. It's the most widely used optimizer in deep learning because it converges fast and works well by default.

PRACTICAL 2: CNN for Fashion MNIST Classification

🎯 Aim / Objective

To build a Convolutional Neural Network (CNN) using Keras to classify grayscale 28x28 images of clothing items from the Fashion MNIST dataset into 10 categories such as T-shirts, trousers, sneakers, bags, etc.

📚 Theory

What is a Convolutional Neural Network (CNN)?

A CNN is a specialized deep learning architecture designed primarily for image data. Unlike a regular ANN, a CNN uses convolution operations to automatically learn spatial hierarchies of features (edges → textures → shapes → objects). CNNs share weights across spatial locations, making them parameter-efficient and translation-invariant.

Key Layers in a CNN

- **Conv2D:** Conv2D (Convolutional Layer): Applies learnable filters (kernels) across the image. Each filter detects a specific feature like horizontal edges, corners, etc. Output is called a Feature Map. Formula: (Feature Map) = Input ★ Filter + Bias.
- **MaxPooling:** MaxPooling2D: Reduces the spatial dimensions of feature maps by taking the maximum value in each pool window (e.g., 2x2). This reduces computation and provides spatial invariance (small shifts in the image don't change features much).
- **Dropout:** Dropout: Randomly sets a fraction of neurons to 0 during training (rate=0.3 means 30% are dropped). Prevents overfitting by forcing the network to learn redundant representations.
- **Flatten:** Flatten: Converts the 2D feature maps into a 1D vector so it can be passed to fully connected Dense layers.
- **Dense:** Dense: Fully connected layer where each neuron connects to all neurons in the previous layer. Used after flattening for classification.

Activation Functions

- ReLU in hidden layers: f(x) = max(0,x). Introduces non-linearity; avoids vanishing gradient.
- Sigmoid in output (10 units): Squashes each output to 0-1 range independently. Used with sparse_categorical_crossentropy for multi-class classification.

Fashion MNIST Dataset

Fashion MNIST is a dataset of 70,000 grayscale images (60,000 train + 10,000 test), each 28x28 pixels. It has 10 classes: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle boot.

Label	Class Name
0	T-shirt/top
1	Trouser
2	Pullover
3	Dress
4	Coat

5	Sandal
6	Shirt
7	Sneaker
8	Bag
9	Ankle boot

🔢 Step-by-Step Code Explanation

Step 1: Load & Preprocess Data

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

train_df = pd.read_csv('fashion-mnist_train.csv')  # 60,000 rows
test_df  = pd.read_csv('fashion-mnist_test.csv')   # 10,000 rows

# Extract features and reshape to (N, 28, 28, 1)
x_train = train_df.iloc[:,1:].to_numpy().reshape([-1, 28, 28, 1])
x_train = x_train / 255   # Normalize pixel values to [0,1]
y_train = train_df.iloc[:,0].to_numpy()

x_test = test_df.iloc[:,1:].to_numpy().reshape([-1, 28, 28, 1])
x_test = x_test / 255
y_test = test_df.iloc[:,0].to_numpy()
```

Each image is 28x28=784 pixels stored as a flat row in the CSV. We reshape to (N,28,28,1) where 1 is the channel (grayscale). Dividing by 255 normalizes pixels from [0,255] to [0,1], which helps training.

Step 2: Visualize Sample Images

```
class_names = ['T-shirt/top','Trouser','Pullover','Dress','Coat',
               'Sandal','Shirt','Sneaker','Bag','Ankle boot']

plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.imshow(x_train[i], cmap=plt.cm.binary)
    plt.xlabel(class_names[y_train[i]])
plt.show()
```

Step 3: Build the CNN Model

```
from keras.models import Sequential
from keras.layers import Dense, Conv2D, Flatten, MaxPooling2D, Dropout

model = Sequential()

# Convolutional block: extract features
model.add(Conv2D(filters=64, kernel_size=(3,3), input_shape=(28,28,1),
                 activation='relu'))
model.add(MaxPooling2D(pool_size=(2,2)))   # Reduce size: 26x26 -> 13x13
model.add(Dropout(rate=0.3))               # Prevent overfitting
```

```
# Fully connected block: classify
model.add(Flatten())                        # 13x13x64 -> 10816 neurons
model.add(Dense(units=32, activation='relu'))
model.add(Dense(units=10, activation='sigmoid'))  # 10 classes

model.compile(loss='sparse_categorical_crossentropy',
              optimizer='adam', metrics=['accuracy'])
model.summary()
```

Step 4: Train & Evaluate

```
model.fit(x_train, y_train, epochs=50, batch_size=1200,
          validation_split=0.05)

evaluation = model.evaluate(x_test, y_test)
print(f'Accuracy: {evaluation[1]}')

# Predict probabilities and get class with highest probability
y_probas = model.predict(x_test)
y_pred   = y_probas.argmax(axis=-1)

from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

📝 Example

Practical Example — Fashion MNIST

A 28x28 grayscale image of an ankle boot is fed to the CNN. The Conv2D layer applies 64 filters, each detecting features like the curved sole or strap patterns. After MaxPooling, the key features are retained at reduced resolution. After Flatten and Dense layers, the output neuron for class 9 (Ankle boot) has the highest value, so the model predicts 'Ankle boot'. The classification report shows precision, recall, and F1-score for each of the 10 clothing categories.

🗣️ Oral Questions & Answers — Practical 2

Q: What is a CNN and why is it used for images?

A: A CNN (Convolutional Neural Network) is a type of deep learning model that uses convolution operations to learn spatial features in images. It's used for images because it automatically detects features like edges, textures, and shapes using shared-weight filters, making it efficient and accurate for image tasks.

Q: What does Conv2D do?

A: Conv2D applies a set of learnable filters (kernels) across the input image using convolution. Each filter slides across the image and produces a feature map that highlights specific patterns. With 64 filters of size 3x3, it produces 64 feature maps detecting different features.

Q: What is MaxPooling and why use it?

A: MaxPooling reduces the spatial size of feature maps by taking the maximum value in each pool window (2x2 in our case). This reduces computation, prevents overfitting, and provides

translation invariance — small shifts in the input don't change the output much.

Q: What is Dropout and why is it important?

A: Dropout randomly deactivates a fraction of neurons during training (rate=0.3 means 30%). This prevents overfitting by making the network learn robust, redundant representations. During testing/inference, Dropout is turned off automatically by Keras.

Q: What is sparse_categorical_crossentropy?

A: It's a loss function for multi-class classification when labels are integers (0,1,2,...9). It calculates the negative log probability of the correct class. 'Sparse' means we pass integer labels directly instead of one-hot encoded vectors.

Q: What is the classification report?

A: It shows Precision (of all predicted positives, how many were actually positive), Recall (of all actual positives, how many were predicted), F1-Score (harmonic mean of precision and recall), and Support (number of samples per class). It gives a detailed per-class performance summary.

Q: Why do we divide pixel values by 255?

A: Pixel values range from 0 to 255. Dividing by 255 normalizes them to [0,1]. Neural networks train better with small, uniform input values. Large inputs cause large activations which destabilize training.

PRACTICAL 3: RNN for Google Stock Price Prediction

🎯 Aim / Objective

To build a Recurrent Neural Network (RNN) using Keras with SimpleRNN layers to predict the future opening price of Google stock using historical stock price data. This demonstrates how RNNs handle sequential, time-series data.

📚 Theory

What is a Recurrent Neural Network (RNN)?

A Recurrent Neural Network (RNN) is a type of neural network designed for sequential data. Unlike feedforward networks (ANN, CNN), RNNs have connections that loop back, giving them a 'memory' of past inputs. At each time step t, the output depends not just on the current input but also on the previous hidden state (memory from past steps).

RNN Formula: h_t = tanh(W_h * h_(t-1) + W_x * x_t + b)

Where: h_t = current hidden state, h_(t-1) = previous hidden state, x_t = current input, W = weight matrices, b = bias

Why RNN for Stock Price Prediction?

- Stock prices are time-series data — each day's price depends on previous days.
- RNNs maintain internal state (memory) across time steps, making them ideal for sequential patterns.
- We use a sliding window of 60 days to predict the next day's price.

Key Concepts in Practical 3

- **MinMaxScaler:** MinMaxScaler: Scales data to [0,1] range. Formula: x_scaled = (x - x_min)/(x_max - x_min). Prevents large values from dominating training.
- **Time Window:** Time Window (60): We take 60 consecutive days of stock prices as input (X) to predict day 61's price (Y). This is the sequence length.
- **Tanh:** Tanh Activation: f(x) = (e^x - e^-x)/(e^x + e^-x). Outputs values between -1 and +1. Helps with gradient flow in RNNs.
- **return_sequences:** return_sequences=True: When True, the RNN layer returns the hidden state at every time step (full sequence). When False, only the last time step's output is returned. Required for stacking multiple RNN layers.
- **Dropout:** Dropout (0.2): Applied after each RNN layer to prevent overfitting. 20% of neurons are randomly dropped during training.
- **inverse_transform:** inverse_transform: After prediction, we reverse the MinMaxScaler to get actual price values from the scaled predictions.

Architecture of the RNN Model

Layer	Units	Details
SimpleRNN 1	50	return_sequences=True, tanh, input=(60,1)
Dropout	—	rate=0.2
SimpleRNN 2	50	return_sequences=True, tanh

Dropout	—	rate=0.2
SimpleRNN 3	50	return_sequences=True, tanh
Dropout	—	rate=0.2
SimpleRNN 4	50	return_sequences=False (last output only)
Dropout	—	rate=0.2
Dense	1	Output — predicted price

🔢 Step-by-Step Code Explanation

Step 1: Load & Split Data

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

df = pd.read_csv('Google Stock Prices Dataset.csv')
# shuffle=False is CRITICAL for time-series data
train_df, test_df = train_test_split(df, test_size=0.2, shuffle=False)

train = train_df.loc[:, ['Open']].values   # Use only 'Open' price column
```

For time-series data, we must NOT shuffle the data. The order of records is crucial because each day's price depends on the previous days.

Step 2: Feature Scaling

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
train_scaled = scaler.fit_transform(train)  # Scale to [0,1]
```

Step 3: Create Sequences (Sliding Window)

```
x_train = []
y_train = []
time = 60   # Look back 60 days to predict the next day

for i in range(60, train_scaled.shape[0]):
    x_train.append(train_scaled[i-60:i, 0])   # Days i-60 to i-1
    y_train.append(train_scaled[i, 0])          # Day i

x_train = np.array(x_train)
y_train = np.array(y_train)

# Reshape to (samples, time_steps, features)
x_train = np.reshape(x_train, (x_train.shape[0], x_train.shape[1], 1))
```

This creates a sliding window. For each day i (from day 60 onward), we take the previous 60 days as input and day i as output. The reshape adds a 'features' dimension (1 feature: Open price).

Step 4: Build the RNN Model

```
from keras.models import Sequential
```

```
from keras.layers import Dense, SimpleRNN, Dropout

model = Sequential()

model.add(SimpleRNN(units=50, activation='tanh',
    return_sequences=True, input_shape=(x_train.shape[1], 1)))
model.add(Dropout(0.2))

model.add(SimpleRNN(units=50, activation='tanh', return_sequences=True))
model.add(Dropout(0.2))

model.add(SimpleRNN(units=50, activation='tanh', return_sequences=True))
model.add(Dropout(0.2))

model.add(SimpleRNN(units=50))      # Last RNN: return_sequences=False
model.add(Dropout(0.2))

model.add(Dense(units=1))           # Output layer

model.compile(optimizer='adam', loss='mse')
model.summary()
```

Step 5: Prepare Test Data & Predict

```
# Combine train+test to get the last 60 days before test period
data = pd.concat((train_df['Open'], test_df['Open']), axis=0)
test_input = data.iloc[len(data) - len(test_df) - time:].values
test_input = test_input.reshape(-1, 1)
test_scaled = scaler.transform(test_input)   # Use same scaler

x_test = []
for i in range(time, test_scaled.shape[0]):
    x_test.append(test_scaled[i-time:i, 0])
x_test = np.array(x_test)
x_test = np.reshape(x_test, (x_test.shape[0], x_test.shape[1], 1))

# Predict and inverse transform to get actual prices
y_pred = model.predict(x_test)
y_pred = scaler.inverse_transform(y_pred)

# Plot real vs predicted
plt.plot(y_test, color='red', label='Real price')
plt.plot(y_pred, color='blue', label='Predicted price')
plt.title('Google Stock Price Prediction')
plt.legend()
plt.show()
```

📝 Example

Practical Example — Google Stock Price

Suppose Google's opening prices for the last 60 days are between $100-$150. After MinMaxScaling, these become values between 0 and 1. The RNN processes this sequence day by day, maintaining memory through its hidden state. After 4 stacked SimpleRNN layers and a Dense output, it predicts the 61st day's scaled price ≈ 0.78. After inverse_transform, this becomes ≈ $139. The real price was $142, showing a close prediction. The final graph plots red (real) vs blue (predicted) lines showing how closely the RNN tracks the stock trend.

🗣️ Oral Questions & Answers — Practical 3

Q: What is an RNN and how is it different from ANN?

A: An RNN (Recurrent Neural Network) has loops in its connections, allowing it to maintain a memory of past inputs. An ANN processes each input independently with no memory. RNNs are designed for sequential/time-series data like text, audio, and stock prices where order and history matter.

Q: What is the sliding window approach? Why 60 time steps?

A: We take 60 consecutive days of stock prices as a single input sample to predict the next day. The window 'slides' — first it's days 1-60 predicting day 61, then days 2-61 predicting day 62, and so on. 60 represents roughly 3 months of trading days, capturing meaningful short-term trends.

Q: Why MinMaxScaler instead of StandardScaler for this practical?

A: MinMaxScaler scales data to [0,1] which works well with RNNs that use tanh activation (output range -1 to 1). It preserves the relative differences in stock prices. StandardScaler (mean=0, std=1) can produce negative values, which is less natural for price data that's always positive.

Q: What does return_sequences=True mean?

A: When return_sequences=True, the RNN layer returns the hidden state at EVERY time step (shape: batch, timesteps, units). This is required when stacking multiple RNN layers — each needs to receive a full sequence. The LAST RNN layer uses return_sequences=False to output only the final time step.

Q: Why use 4 stacked RNN layers?

A: Stacking multiple RNN layers creates a deeper network that can learn more complex temporal patterns. The lower layers learn short-term patterns (day-to-day fluctuations) while higher layers capture longer-term trends. Dropout between layers prevents overfitting.

Q: Why do we use scaler.transform (not fit_transform) on test data?

A: We must use the SAME scaling parameters (min, max) from training data on test data. Using fit_transform on test data would compute new min/max from test data, causing data leakage and inconsistent scaling. Always fit the scaler only on training data.

Q: What is tanh activation in RNN and why is it used?

A: Tanh (hyperbolic tangent) outputs values between -1 and +1. It's used in RNNs because it helps control the magnitude of the hidden state (memory), preventing it from growing too large across time steps. It's better than ReLU for RNNs because ReLU can cause the hidden state to explode over many time steps.

QUICK REVISION SUMMARY — All 3 Practicals

Topic	DL-1: ANN	DL-2: CNN	DL-3: RNN
Task	Regression	Classification	Time-series
Dataset	Boston Housing	Fashion MNIST	Google Stock
Data Type	Tabular CSV	28x28 Images	Sequential prices
Model	ANN (Dense)	CNN (Conv2D)	RNN (SimpleRNN)
Key Layers	Dense(128→64→1)	Conv2D+Pool+Dense	4x SimpleRNN+Dense
Activation (hidden)	ReLU	ReLU	tanh
Activation (output)	Linear (regression)	Sigmoid (10 class)	Linear
Loss Function	MSE	Sparse Categorical CE	MSE
Optimizer	Adam	Adam	Adam
Scaler	StandardScaler	Divide by 255	MinMaxScaler
Metric	MSE, MAE	Accuracy	MSE
Epochs	100	50	100

Good luck with your practical exam and oral! 🎓